

DEHURDLE

Software Engineer Technical Assignment

Estimated Time: 4-6 hrs

This assignment evaluates your practical engineering skills across the full MERN stack, along with your comfort using developer tooling, the command line, Git workflows, and AWS basics. There are no trick questions. We value clean, working code and clear decision-making over perfection. Please read all tasks before starting.

PART A: BACKEND (NODE.JS + EXPRESS + MONGODB)

Important: Use a public GitHub repository and commit incrementally.

Task 1: Build a Task Manager REST API

Node.js · Express · MongoDB · Mongoose

Build a task management REST API with the following endpoints:

- POST /tasks: Create a task with title, description, status (todo / in-progress / done), and dueDate
- GET /tasks: List all tasks, with filter support by status query param
- PATCH /tasks/:id: Update task status or any task detail
- DELETE /tasks/:id: Delete a task

Use MongoDB with Mongoose for persistence. Add input validation and return meaningful error responses. Include a .env example file listing all required environment variables.

Task 2: Add JWT-based Authentication

Auth · JWT · Middleware · bcrypt

Protect the task endpoints with JWT authentication:

- Implement POST /auth/register and POST /auth/login
- Store hashed passwords using bcrypt or equivalent
- Validate JWT via middleware on all /tasks routes
- Each user should only see and manage their own tasks

PART B: FRONTEND (REACT)

Task 3: Build a Task Dashboard UI

React · State Management · API Integration

Build a React frontend that integrates with your API:

- Login and Register screens
- Dashboard listing tasks with a filter by status
- Ability to create, update status, and delete tasks inline
- Handle loading, error, and empty states gracefully

We are not evaluating design polish; the UI should be functional and usable. You may use any UI library (or none). Briefly justify your state management approach in the README.

PART C: CLI, GIT & AWS (TOOLING PROFICIENCY)

Task 4 Git Workflow

Git · Branching · Commit Discipline

Your repository should demonstrate intermediate Git usage:

- Use a feature branching strategy (e.g. feature/auth, feature/tasks-api) merged into main
- Write meaningful commit messages, what changed and why
- Include a .gitignore that excludes node_modules, .env, and build artifacts
- Squash or rebase noisy commits before final submission, keep history clean

Task 5 CLI Setup Script

Bash · Shell Scripting · Developer Experience

Write a shell script setup.sh that:

- Installs dependencies for both frontend and backend (npm install)
- Copies .env.example to .env if no .env already exists
- Prints a friendly message confirming the app is ready to run

Bonus: add a --seed flag that populates the database with a few sample tasks.

Task 6: Deploy Backend to AWS

AWS EC2 · PM2 or Docker · Cloud Deployment

Deploy your Node.js backend to AWS. Minimum requirements:

- Spin up an EC2 instance (free tier is fine) and deploy the API
- Use PM2, systemd, or Docker to keep the process running
- Share the public URL of a working endpoint in your README (e.g. GET /tasks returning a 401)
- Describe your setup briefly: instance type, security group rules, and how you connected via SSH

If AWS access is a constraint, deploy to a free alternative (Railway, Render, Fly.io) and document your reasoning; we care about the deployment thinking, not the platform.

SUBMISSION CHECKLIST

Share a link to your GitHub repository containing the following:

Backend and frontend code in clearly named folders

README.md with local setup instructions, tech decisions, and deployed API URL

.env.example for all required environment variables

setup.sh script (and --seed bonus if attempted)

Deployed backend URL (EC2 or documented alternative)

Clean git history with feature branches merged into main